

Tacit-AR: 암묵적 지식 기반 제스처-터치 통합 인터페이스 시스템

Tacit Knowledge의 축적 및 AR 시스템으로의
일관된 UI/UX 이관

반세현 (20213073, AI융합학부)

목차

연구의 전체 구조

- 1 연구 동기
- 2 핵심 개념: Tacit Knowledge와 제스처 일관성
- 3 시스템 설계: 아키텍처 / 하드웨어 구성
- 4 데이터 수집: Hand / Android 이벤트
- 5 연구 방법론의 진화
- 6 실험 케이스 분석 (Case 1~6)
- 7 실험 단계별 비교
- 8 시연

연구 동기

인터페이스 진화에 따른 사용자 경험 리셋 문제

■ 연구의 필요성

- 터치 → 제스처 → AR/VR로의 인터페이스 진화
- 사용자가 축적한 암묵적 지식(tacit knowledge) 손실
- 새로운 학습 비용 발생 → 사용성 저하

■ 연구 목표

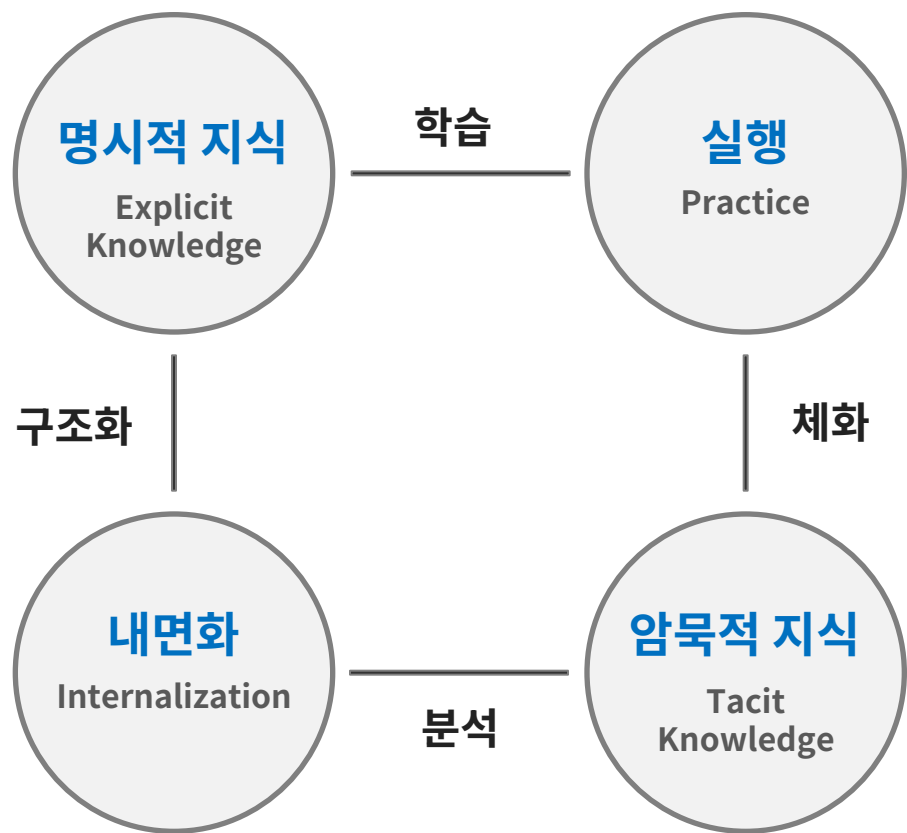
- 사용자의 암묵적 제스처 지식을 정량화
- 터치 → AR 인터페이스 전환 시 UI/UX 일관성 유지
- 암묵적 제스처 지식을 모델 학습



Tacit Knowledge의 디지털 전이를 통한 일관된 UI/UX 경험 실현

Tacit Knowledge 개념

Polanyi의 암묵적 지식 이론과 제스처에서의 응용



Polanyi (1966)의 Tacit Knowledge (암묵적 지식)

■ 명시적 지식 vs 암묵적 지식

- 명시적 지식 (Explicit Knowledge): 구조화·분석·문서화 가능
- 암묵적 지식 (Tacit Knowledge): 체화·내면화된 수행, 설명은 어려우나 자연스럽게 실행

■ 제스처에서의 암묵적 지식

- 사용자는 자신의 제스처 패턴을 설명할 수 없지만 자연스럽게 수행
- 반복 사용을 통해 무의식적으로 형성되는 습관
- 명시적 학습 없이도 일관된 패턴 출현

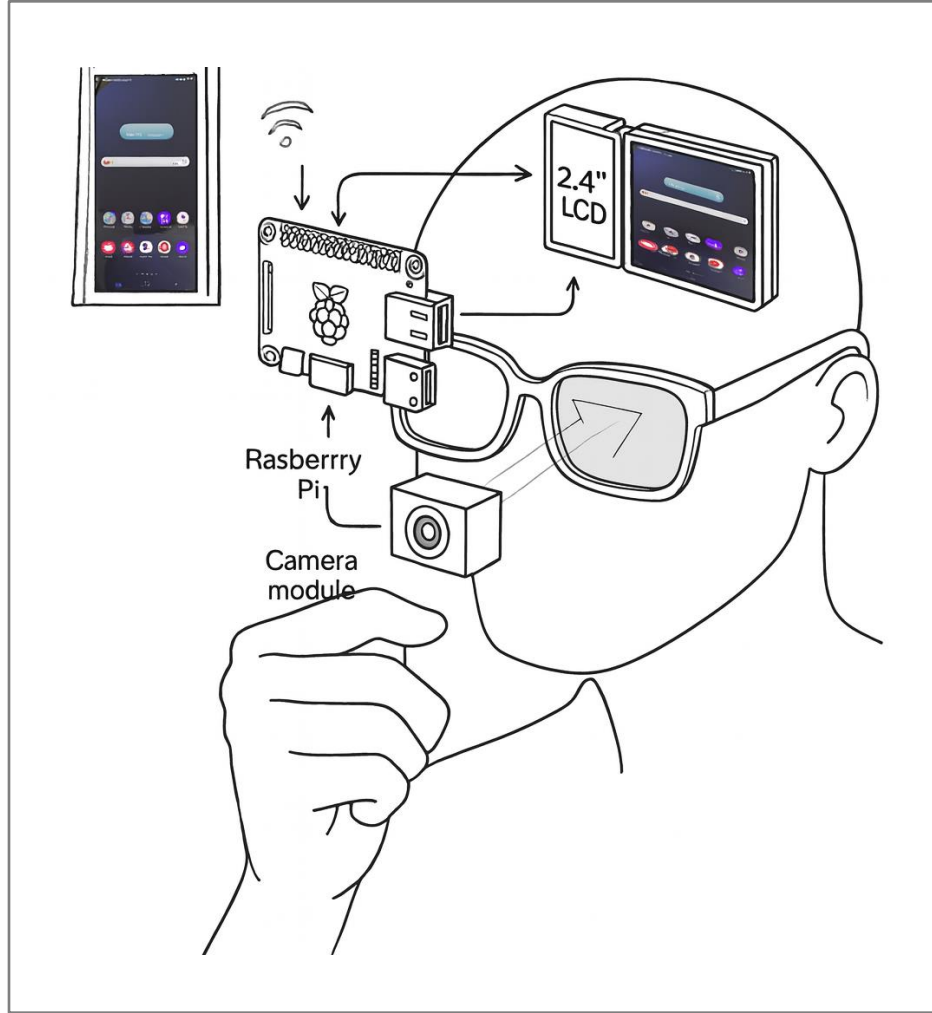


제스처 일관성 (Gesture Consistency)

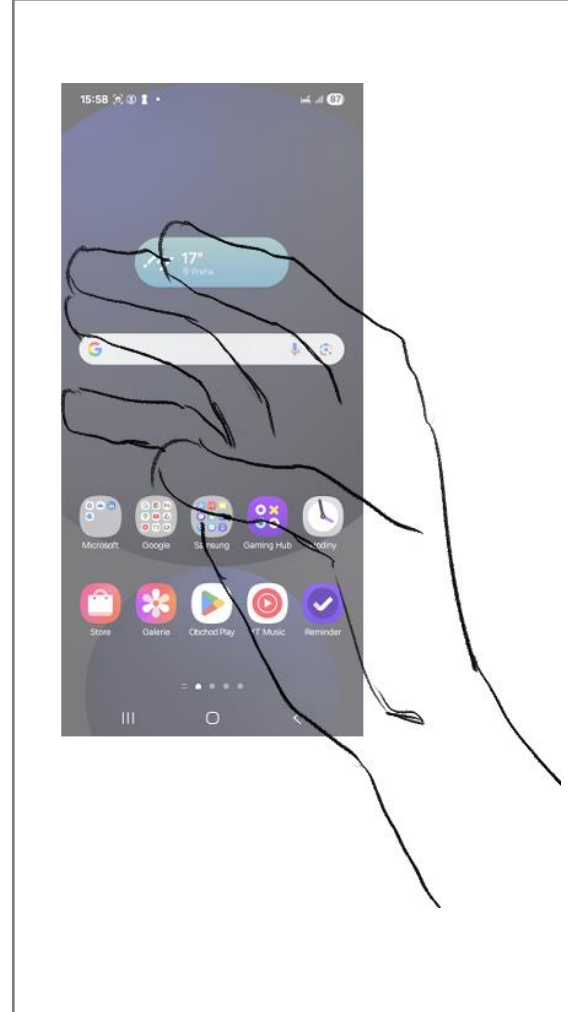
동일한 제스처를 반복 수행할 때 움직임 패턴의 유사도를 측정.
유사도가 높을수록 사용자가 해당 제스처에 대한 일관된 암묵적 지식을 형성했음을 의미

시스템 아키텍처

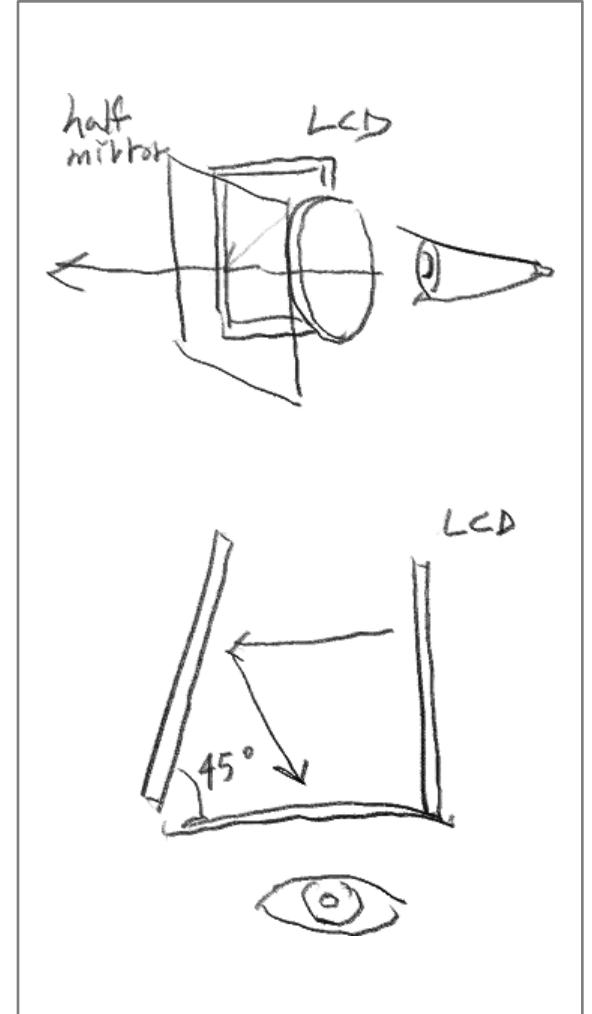
손 제스처 추적 및 AR 오버레이 시스템 구성



사용자의 모습



사용자 시점

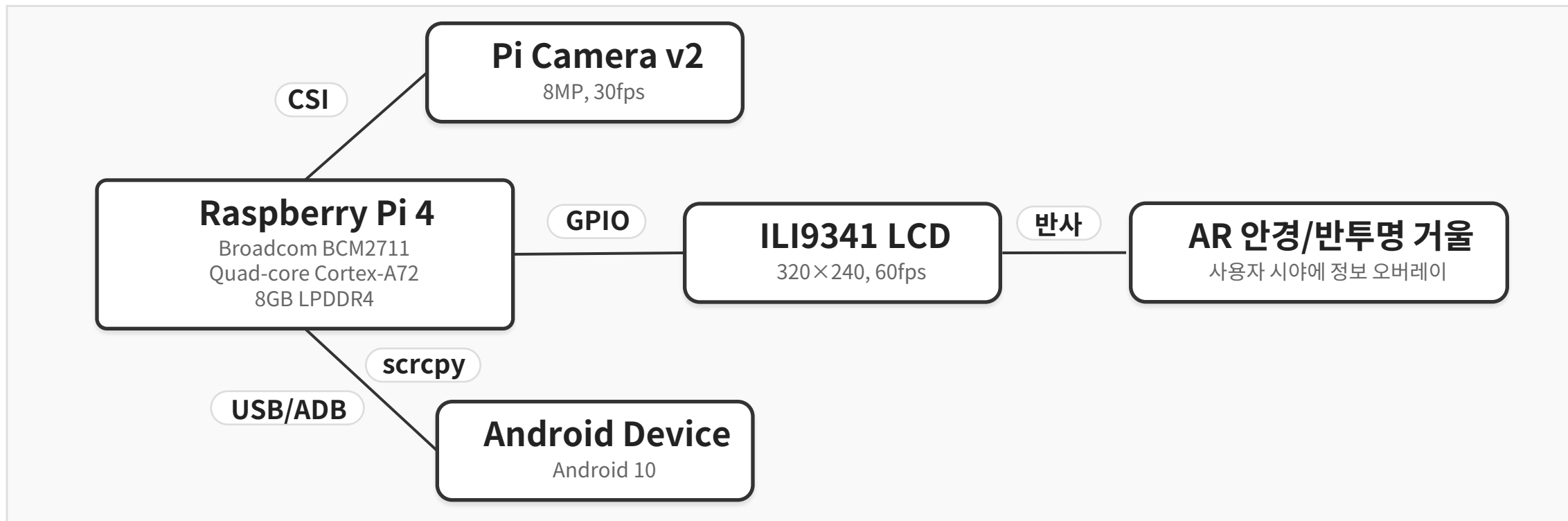


AR-오버레이 원리

하드웨어 구성

시스템 컴포넌트 및 데이터 흐름

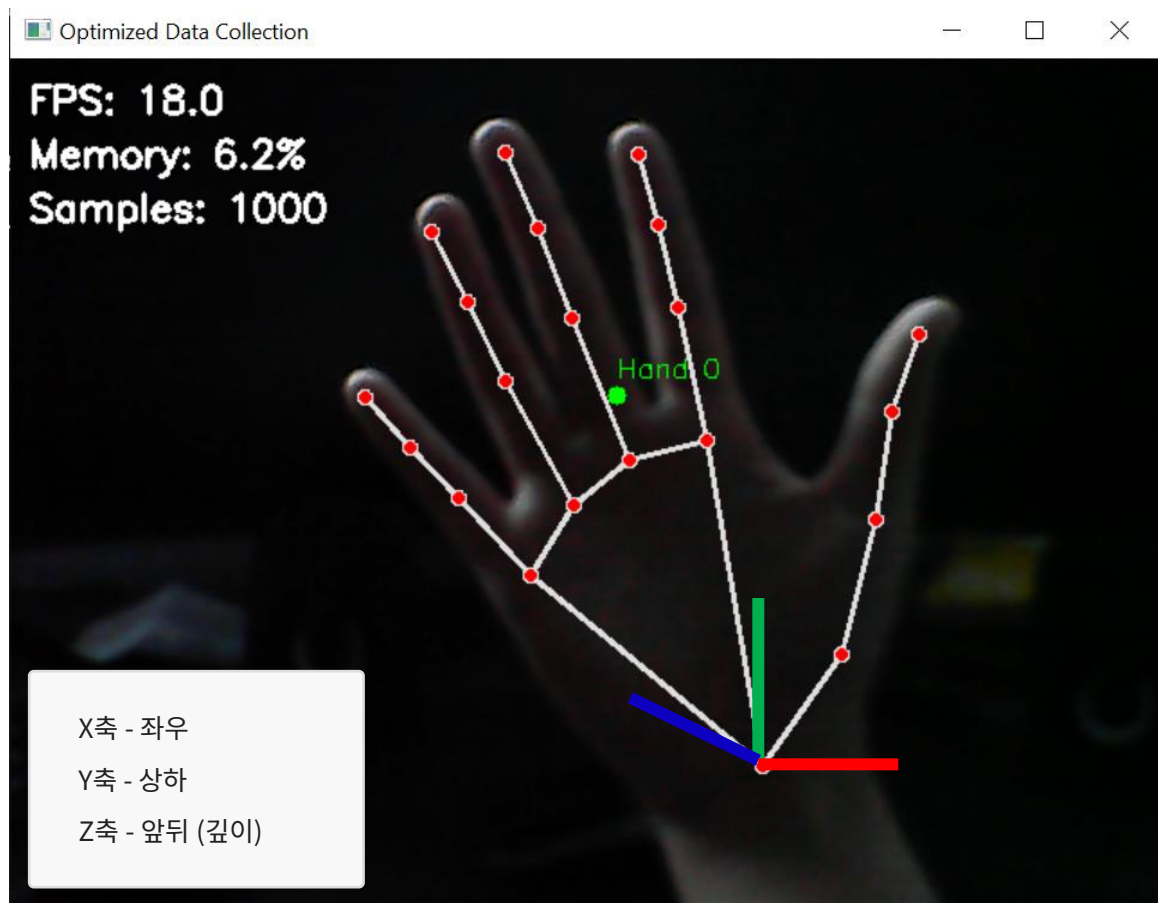
■ 하드웨어 시스템 구성



- 라즈베리파이를 중심으로 카메라(입력), LCD(출력), 안드로이드 기기(제어)가 연결
- 손 제스처 영상 데이터는 카메라→라즈베리파이→안드로이드 기기 흐름으로 처리
- 시각적 피드백은 LCD→반투명 거울→사용자 시야 경로로 전달되며, 안드로이드 화면은 scrcpy를 통해 LCD로 미러링

데이터 수집: Hand

MediaPipe Hands를 이용한 손 랜드마크 데이터 수집



Hand motion 데이터

- MediaPipe Hands 라이브러리 사용
- 21개 랜드마크 $\times$ (x, y, z) = 63개 피쳐
- 샘플링 속도: 30 FPS

이벤트 데이터 예시

timestamp	x_0	y_0	z_0	x_1	y_1	z_1	x_2	y_2
1.763E+09	0.796338	0.970338	4.87E-07	0.806972	0.825662	-0.04423	0.760606	0.680797
1.763E+09	0.811424	0.988923	6.21E-07	0.81271	0.835477	-0.04453	0.767194	0.686323
1.763E+09	0.821626	0.996849	4.82E-07	0.810907	0.844547	-0.03561	0.75616	0.701707
1.763E+09	0.81368	0.99646	5.41E-07	0.800935	0.831972	-0.03315	0.745854	0.683927
1.763E+09	0.807676	0.964458	5.69E-07	0.792311	0.80437	-0.03688	0.736138	0.659884
1.763E+09	0.801496	0.952378	5.23E-07	0.784504	0.788175	-0.0349	0.725763	0.640308
1.763E+09	0.793146	0.93956	5.28E-07	0.775302	0.77598	-0.03585	0.718548	0.627072
1.763E+09	0.792284	0.924499	5.07E-07	0.771683	0.762002	-0.03375	0.712128	0.613614

데이터 수집: Android 이벤트

이벤트 데이터 구조 및 이벤트 타입

이벤트 데이터 구조

필드	데이터 타입	설명
timestamp	Long	이벤트 발생 시간 (millisecond)
type	Integer	이벤트 유형 (예: EV_SYN, EV_KEY, EV_ABS)
code	Integer	이벤트 코드 (예: BTN_TOUCH, ABS_MT_POSITION_X)
value	Integer	이벤트 값 (좌표, 상태 등)

이벤트 데이터 예시

timestamp	raw_line	timestamp	device	type	code	value
1762569759	[79143.208172] /dev/input/event4: 0003 0039 00000487	79143.20817	/dev/input/event4	3	39	487
1762569759	[79143.208172] /dev/input/event4: 0003 0035 0000026b	79143.20817	/dev/input/event4	3	35	0000026b
1762569759	[79143.208172] /dev/input/event4: 0003 0036 00000786	79143.20817	/dev/input/event4	3	36	786
1762569759	[79143.208172] /dev/input/event4: 0003 0030 0000000a	79143.20817	/dev/input/event4	3	30	0000000a
1762569759	[79143.208172] /dev/input/event4: 0001 014a 00000001	79143.20817	/dev/input/event4	1	014a	1
1762569759	[79143.208172] /dev/input/event4: 0001 0145 00000001	79143.20817	/dev/input/event4	1	145	1
1762569759	[79143.208172] /dev/input/event4: 0000 0000 00000000	79143.20817	/dev/input/event4	0	0	0
1762569759	[79143.265393] /dev/input/event4: 0003 0030 0000000b	79143.26539	/dev/input/event4	3	30	0000000b
1762569759	[79143.265393] /dev/input/event4: 0000 0000 00000000	79143.26539	/dev/input/event4	0	0	0
1762569759	[79143.290640] /dev/input/event4: 0003 0030 0000000d	79143.29064	/dev/input/event4	3	30	0000000d

이벤트 타입



EV_SYN (0x0000)

동기화 이벤트. 다중 이벤트의 끝을 표시하며, 하나의 물리적 액션이 완료되었음을 알립니다.



EV_KEY (0x0001)

버튼 이벤트. 터치 이벤트의 시작과 종료를 나타내며, BTN_TOUCH 코드와 함께 사용됩니다.



EV_ABS (0x0003)

절대 좌표 이벤트. ABS_MT_POSITION_X, ABS_MT_POSITION_Y 등의 코드와 함께 터치 위치를 나타냅니다.

연구 방법론의 진화

데이터 이해에 기반한 단계적 접근법

핵심 통찰

Time sequential 문제를 Tabular 문제로 재구성하면
복잡한 DL 없이도 효율적인 ML로 해결 가능

가설 검증 결과

- 가설 1: "복잡한 DL 모델이 항상 더 좋은가?" → ❌
- 가설 2: "적절한 전처리로 문제를 단순화할 수 있는가?" → ✅
- 가설 3: "Sequence 문제를 Timestamp 단일 값 문제로 재정의할 수 있는가?" → ✅

6단계 실험 개요

1) Case 1: Seq2Seq + 절대좌표($63 \times T \rightarrow \text{이벤트} \times T$)

2) Case 2: 손목 기준 상대좌표($15 \times T$)

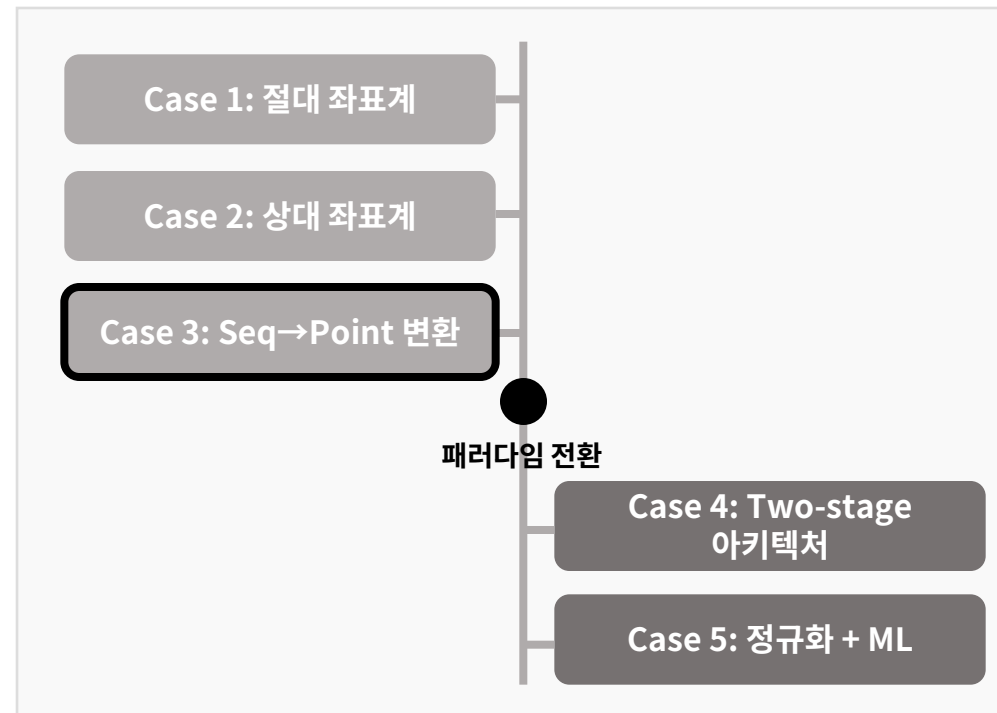
3) Case 3: Seq→Point (Touch 좌표 단일 시점)

4) Case 4: Two-Stage(검출/회귀) 전환점

5) Case 5: Edge 증강(실패)로 문제 재인식

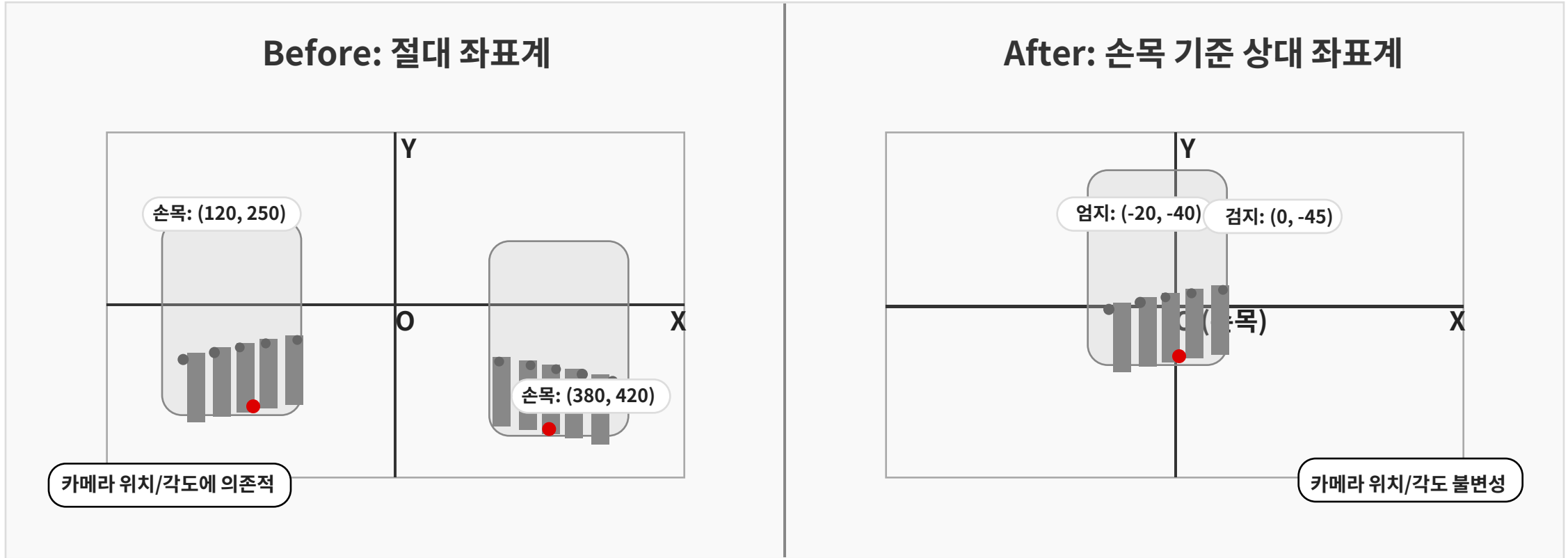
6) Case 6: 값 분포 변환(Yeo-Johnson) + ML

연구 방법론의 진화 과정



Case 1-2: 좌표계 개선

절대 좌표에서 손목 기준 상대 좌표로의 전환



Case 1: 절대 좌표의 문제

- 좌표계 불일치 문제: 카메라 좌표계 $\neq$ 스크린 좌표계
- 손의 위치가 달라지면 동일 제스처도 다른 좌표 생성

Case 2: 손목 기준 상대 좌표

- 변환 공식: $\text{coord_rel} = \text{coord_abs} - \text{wrist}$
- 카메라 각도/거리 변화에 대한 강건성

Case 3: Target 단순화와 문제 발견

Sequence → Point 접근과 Touch Condition의 중요성

■ 접근: Target 단순화

- Target 변경: Sequence → Single Timestamp
- 이전: (type, code, value) × T
- 변경: (timestamp, X, Y) × 1

■ 치명적 문제 발견

- Touch Condition의 부재
- 손이 어디에 있든 Touch 신호 발생

$P(X, Y \mid \text{Hand})$ 는 항상 값을 출력

필요한 것: $P(\text{Touch} \mid \text{Hand}) \times P(X, Y \mid \text{Hand}, \text{Touch}=\text{True})$

- "언제 출력할지" 결정이 먼저 필요함

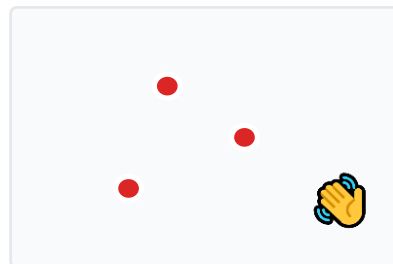
Target 단순화: Sequence → Point

Hand
Sequence
(15×T)

단일 좌표
(X,Y)×1

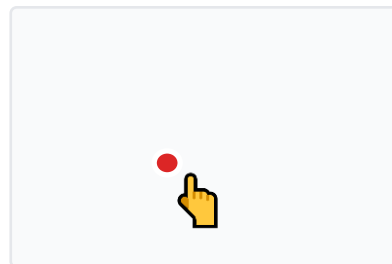
순서 정보 → 단일 시점 예측으로 단순화

False Positive 문제 (80%)



터치 없음 상황

손 위치와 무관하게 터치 예측



터치 상황

좌표는 잘 예측하지만
언제 터치할지 모름

Case 4: Two-Stage 아키텍처

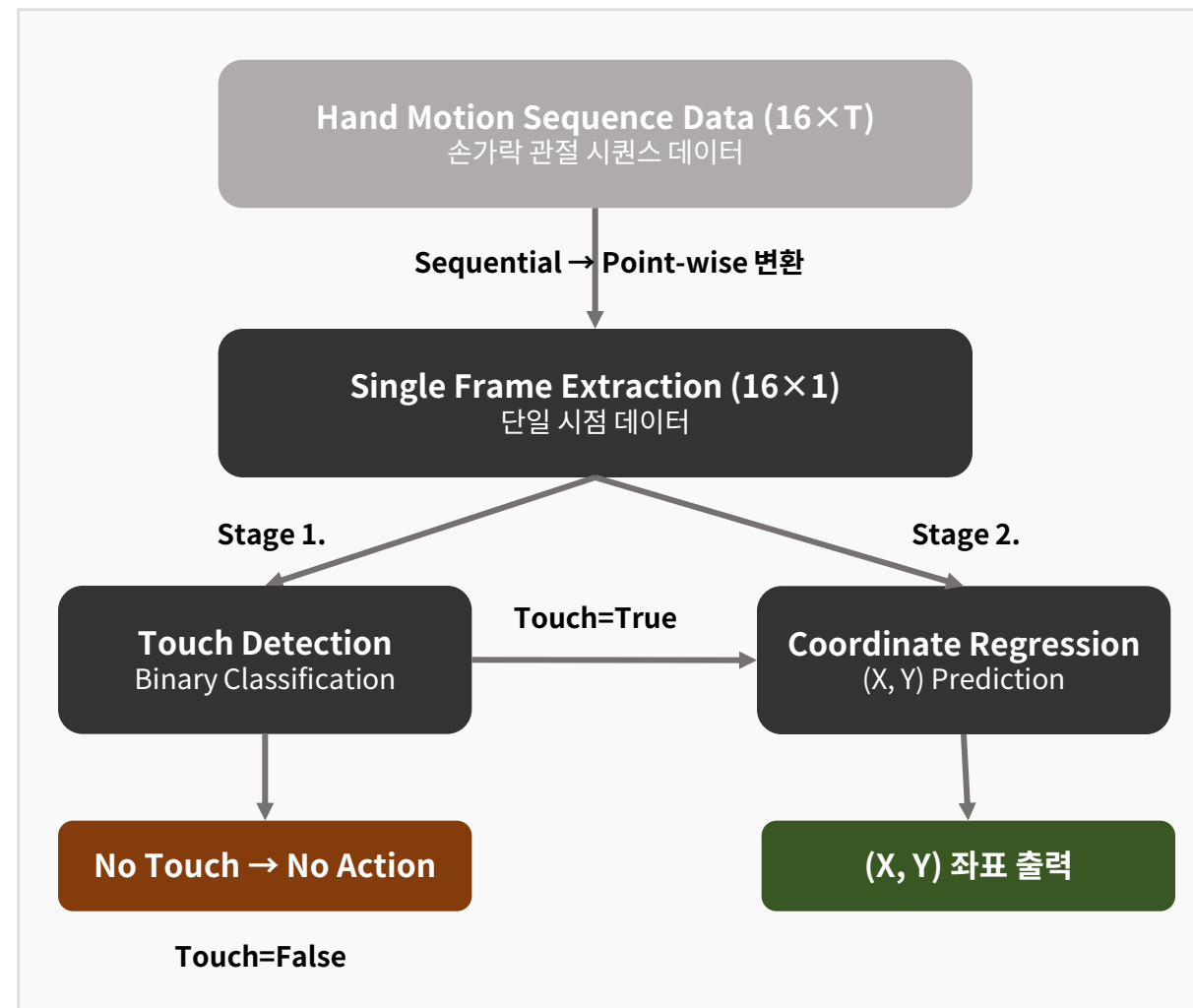
패러다임 전환: Sequential → Point-wise 접근법

패러다임 전환

- Sequential Problem → Point-wise Problem
- "언제 예측할지"를 명확히 분리하는 접근법

Two-Stage 설계

- Stage 1: Touch Detection (Binary Classification)
 - Feature: Thumb joint diff (16×1)
 - Target: Touch 여부 (0 or 1)
 - Model: ML Classifier
- Stage 2: Coordinate Regression (Touch=True일 때만)
 - Feature: Thumb joints (16×1)
 - Target: (X, Y) 좌표
 - Model: DL/ML Regressor



Case 5-6: 최종 솔루션

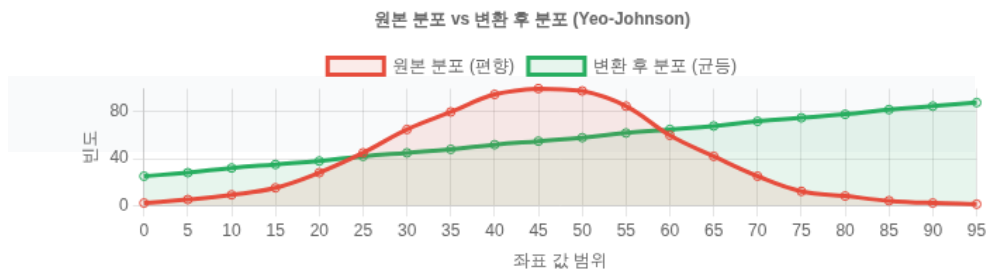
값 분포 변환을 통한 Edge 예측 성능 개선

Case 5: Edge Sample 증강 실패

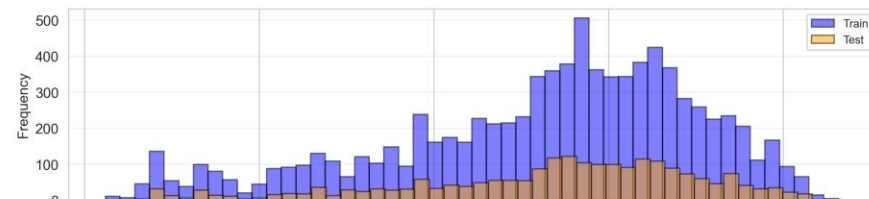
- 가장자리 샘플 3배 증강 시도 → 효과 제한적
- 원인: MSE Loss의 본질적 한계 (다수 샘플 만족)
- 문제 재인식: 샘플 수 불균형 vs 값 분포 편향

Case 6: 값 분포 변환 (최종 솔루션)

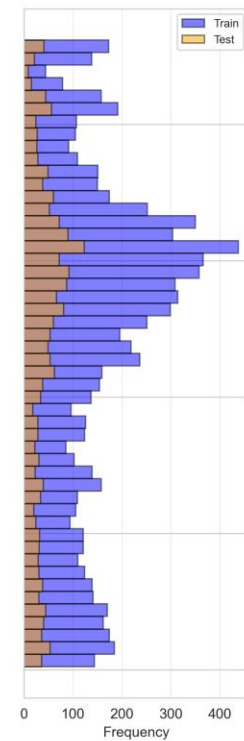
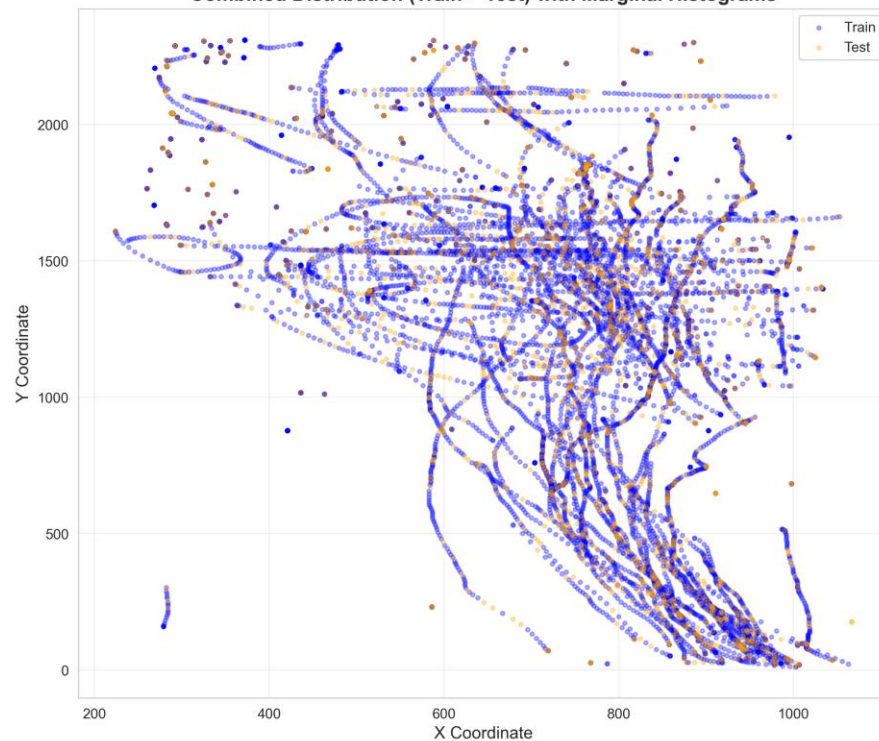
- 좌표 값의 수학적 변환으로 분포 평활화
- 모델 최종 선택: Random Forest (경량, 해석 가능, 빠른 추론)
- 변환 방법: 원본, Log transform, Box-Cox Transform, Yeo-Johnson Transform



편향된 좌표 분포



Combined Distribution (Train + Test) with Marginal Histograms



실험 단계별 비교

ML/DL 모델 비교 및 성능 평가

실험 설계 규모

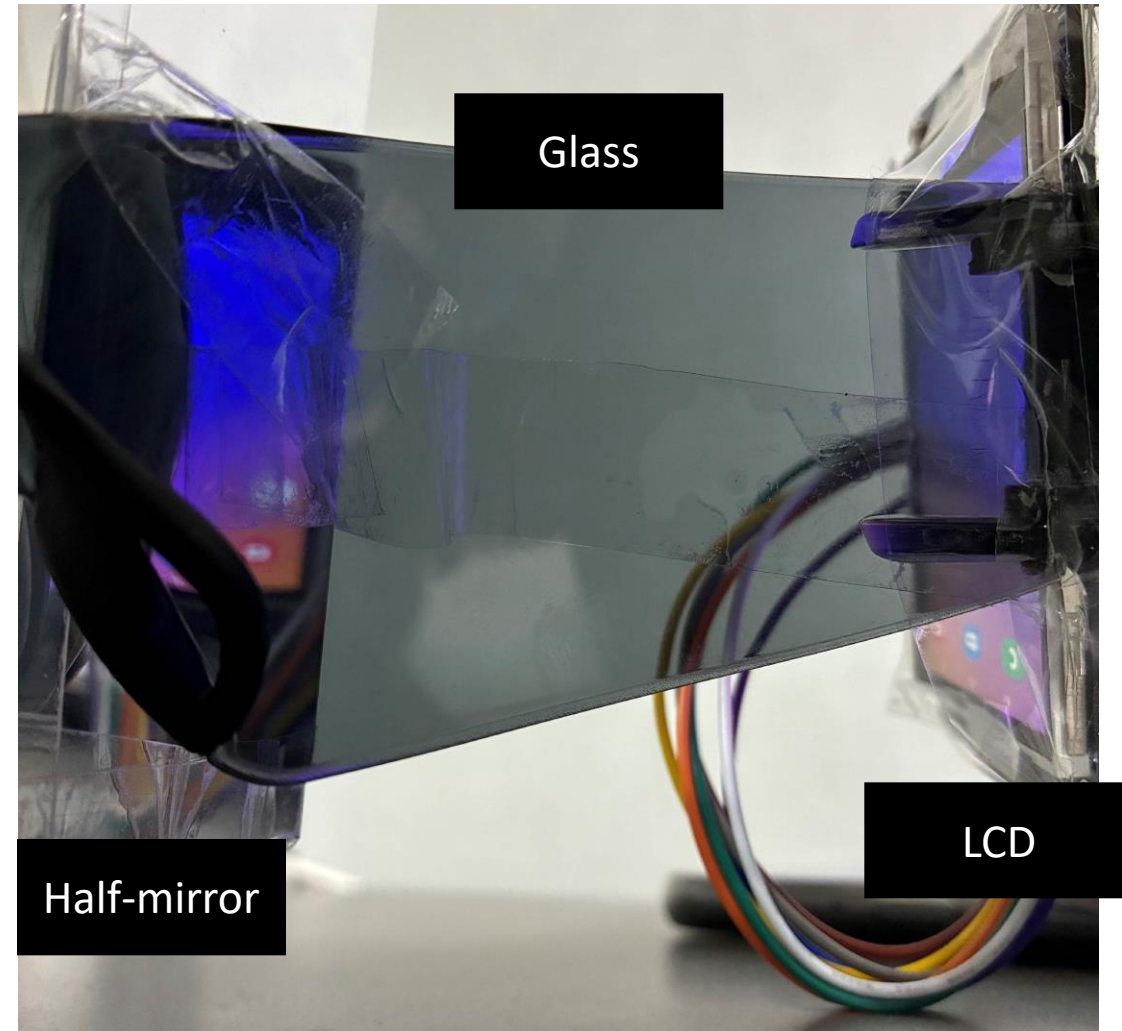
총 실험 수: 156개 = 13개 모델 × 12개 조합	모델당 조합: 12개 = 4 Target Transform × 3 Time Window	ML 모델: 4개 (CatBoost, LightGBM, Random Forest, XGBoost)
DL 모델: 9개 (Transformer, LSTM계열 4개, GRU계열 4개)	Target Transform: None, Log, Box-Cox, Yeo-Johnson	Time Window: 0.05초, 0.1초, 0.2초

모델	아키텍처 유형	주요 특징	성능
ML 모델 (평균 0.393447187)			
CatBoost	Gradient Boosting	범주형 특성 자동 처리, 순열 기반 대안 적합, 과적합 감소	0.5283 (최대), None 변환
LightGBM	Gradient Boosting	히스토그램 기반 알고리즘, GOSS 샘플링, 리프 중심 분할	0.4921, Box-Cox 변환
Random Forest	앙상블 결정 트리	배깅 기법, 특성 랜덤 선택, 오버피팅 방지	0.3826, None 변환
XGBoost	Gradient Boosting	정규화 항 포함, 병렬 처리, 결측치 자동 처리	0.3636, Log 변환
DL 모델 (평균 0.068750973)			
Transformer	Self-Attention 기반	RNN 구조 없이 Self-Attention만 활용, 병렬 처리 효율 우수	0.2811 (DL 최대), None 변환
Attention LSTM	어텐션 메커니즘 RNN	중요 시퀀스 위치에 가중치 부여, 선택적 정보 집중	0.1732, None 변환
Bidirectional LSTM	양방향 순환 신경망	전방/후방 컨텍스트 모두 고려, 시퀀스 전체 정보 활용	0.0453, None 변환
Basic LSTM	Recurrent Neural Network	장기 종속성 포착, 게이트 메커니즘으로 정보 선택적 기억/삭제	0.0327, Box-Cox 변환
MLP	다층 퍼셉트론	완전 연결 레이어, 비선형 활성화 함수, 시퀀스 정보 미활용	0.0125, Log 변환
Bidirectional GRU	양방향 GRU	양방향 정보 흐름, GRU의 효율성 유지	0.0094, None 변환
GRU	Gated Recurrent Unit	LSTM 간소화 버전, 리셋/업데이트 게이트, 계산 효율성 향상	0.0032, None 변환
Conv-LSTM	Convolutional LSTM	컨볼루션 레이어 + LSTM, 공간적 특성 추출 후 시간적 학습	-0.1235, Log 변환
Conv1D-GRU	1D Convolutional GRU	1D 컨볼루션으로 로컬 특징 추출 후 GRU 처리, 경량화 모델	-0.2294, Box-Cox 변환

성능은 R^2 점수로, 값이 높을수록 좋음. ML 모델이 DL 모델보다 평균적으로 우수하며, 특히 CatBoost+None 조합이 최고 성능 달성. Yeo-Johnson 변환은 모든 모델에서 성능 저하 (-0.6522).

시연

실제 프로토타입



시연

실제 프로토타입

